

Fabrication After Tool Failure: Tool-Augmented Agents Assert Values Their Tools Did Not Return

Arham Sethi*
The Shishukunj International School

Arsen Kenzhebayev*
Haileybury Astana

Saanvi Paturi*
UWCSEA East Campus

Vatsal Raina
Apta AI & Spark AI Research

Vyas Raina[†]
Apta AI & Spark AI Research

Ivaxi Sheth
Spark AI Research

Abstract

Tool-augmented language models are evaluated on whether they reach the right answer, not on whether they report honestly when a tool fails to supply one. We isolate this post-failure decision with a benchmark of 1,024 items spanning 16 internal-system domains and eight tool-failure types, in which a tool call is enforced and the returned payload is guaranteed to be unusable. Under a deployment-style system prompt, 14.10% of responses are dishonest: the model either asserts a value the payload cannot support or declines while citing a fabricated policy or capability limit. The rate is governed almost entirely by whether the failure is signalled. When the tool returns `status:error`, dishonesty is absent (0.0%); when it returns `status:ok` with a redacted, corrupted, stale, malformed, empty or truncated value, dishonesty reaches 45.3%. The behaviour is not an artefact of our prompts: it appears under a neutral prompt (10.17%) and under the shipped prompt of every production agent framework we evaluate, reaching 24.67% under CrewAI’s, and none of the nine frameworks we audit specifies what the model should do when a tool fails. Comparing prompt-level defences, we find that the operative variable is not deference to tool output but the absence of a named failure state. Appending a single sentence that requires the model to emit `retrieval_status: OK` or `FAILED` before answering reduces dishonesty from 14.10% to 0.87%, with one item of 688 worsening against 92 improving, and transfers unchanged into three foreign agent scaffolds. The emitted flag is faithful in 99.7–99.9% of declarations, giving a runtime detector that needs only a regular expression.

1 Introduction

Large language models (LLMs) are increasingly given external tools that they may call while generating a response [Schick et al., 2023, Qin et al., 2024, Li et al., 2023]. A model that retrieves a value rather than recalling it is no longer limited by what its parameters encode [Schick et al., 2023, Qin et al., 2024], and the gain is largest for information the model cannot hold at all: live readings, current statuses and records in private systems change after training, so a tool is the only route to them [Li et al., 2023, Patil et al., 2025]. The tool then also supplies the grounding for the answer, which is supposed to rest on the returned payload rather than on the model’s prior.

Tools do not always deliver that grounding. A tool may be unreachable, the model may select the wrong tool or none at all, and the call it constructs may be malformed or carry arguments the tool rejects [Patil et al., 2025, Ross et al., 2025]. These pipeline failures have been studied as such,

*Work done while at Spark AI Research.

[†]Correspondence: vyas@sparkairesearch.com

including recovery from a call that does not succeed [Zhu et al., 2026, Soni, 2026]. We study a different case: the call is well formed, the tool is reached, and the tool still returns nothing the model can use. The model then faces a choice that the pipeline literature does not score: report the failure honestly, or assert an answer the payload cannot support. We call the second outcome *fabrication after tool failure*. It includes newly invented values and *relay*, in which the model repeats corrupted or redacted content or reads missing data as zero or “none.” A related outcome withholds the value but blames a policy, permission, safety or capability restriction that the tool’s declaration does not support, rather than the observed failure [Singh, 2026]. Both outcomes misrepresent the state of the world to the user.

Existing tool-use evaluations [Li et al., 2023, Patil et al., 2025, Zhu et al., 2026, Soni, 2026, Ross et al., 2025] do not separate a fabricated answer from a truthful failure report, since neither returns the requested value. We isolate the post-failure decision with 1,024 items spanning 16 fictional internal-system domains and eight failure types. Each query asks for an exact value available only through a tool; the model is required to call the tool and then receives an injected, unusable payload. Matched items are evaluated under prompt conditions that differ only in the tool-use instruction, and a payload-aware judge classifies each response as an honest report, an unfaithful refusal, a hedged answer or a fabrication.

The behaviour is common, is governed almost entirely by whether the failure is signalled in the tool’s response envelope rather than by the domain or the query, and appears under the shipped prompt of every production framework we evaluate. Comparing prompt-level mitigations, we show that what makes a defence work is not whether it removes the instruction to defer to tool output but whether it names a state the model may occupy instead of answering. The resulting single-sentence defence reduces dishonesty by an order of magnitude and transfers unchanged into three foreign agent scaffolds.

Contributions.

- A benchmark isolating fabrication after *operational* tool failure: 16 domains $\times$ 8 failure types, with an enforced tool call, an unusable payload, and no other route to the requested value, unlike degraded-retrieval settings in which retrieval succeeds and the content is merely poor.
- A payload-aware label set (honest report, unfaithful refusal, hedged, fabrication), validated against human annotation in which the annotator receives the tool’s declared capability, so a stated refusal reason is checked rather than assumed.
- An audit of nine production agent frameworks, each pinned to a commit, showing that none specifies model behaviour under tool failure, and an end-to-end evaluation of three of their shipped prompts.
- A decomposition along two axes not previously separated: whether the failure is signalled, which dominates, and the framework prompt driving the agent.
- A prompt-level defence, selected on a pre-registered development split and validated on held-out domains, that cuts dishonesty by an order of magnitude, transfers unmodified into three foreign scaffolds, and emits a machine-readable flag usable as a runtime detector.

2 Related Work

2.1 Tool-use benchmarks

Evaluation of tool-augmented agents has emphasised task completion rather than faithful reporting of tool outcomes: following Toolformer [Schick et al., 2023], ToolLLM [Qin et al., 2024], API-Bank [Li et al., 2023], BFCL [Patil et al., 2025] and agentic suites [Mialon et al., 2024, Liu et al., 2024, Zhou et al., 2024, Yao et al., 2024] score planning, invocation and final-answer correctness, so a fabricated output and a truthful failure report score the same whenever both yield an incorrect answer. WebArena models infeasibility as a property of the task rather than of a failed call, and BFCL asks whether a tool should be invoked rather than whether its outcome is reported faithfully. The one audit of deployed agent frameworks we are aware of evaluates architectural containment rather than failure handling [Hossain et al., 2026]. Our benchmark scores what remains once the tool has been called and the call was well formed: the truthfulness of the model’s report.

2.2 Tool failure and degraded retrieval

A related literature studies grounding under degraded retrieved context. RGB formalises *negative rejection*, requiring abstention when retrieved documents are irrelevant [Chen et al., 2024]; FaithEval reports unsupported answers from evidence-free contexts [Ming et al., 2025]; ClashEval shows that perturbed retrieval can override correct prior knowledge [Wu et al., 2024]; and retrieval augmentation can underperform direct generation when retrieval is poor [Wang et al., 2025], motivating mitigations [Yoran et al., 2024, Zhou et al., 2023]. These studies concern degraded content after successful retrieval; an operational tool failure returns no usable content at all. Recent work injects such failures directly: Zhu et al. [2026] evaluate recovery from corrupted outputs rather than truthful reporting, Soni [2026] studies fabrication when successful tool outputs contradict memorised knowledge, and Liu et al. [2026] measure abstention from acting rather than truthful reporting. Closest to our work, Singh [2026] injects silent failures across tool stubs, reports widespread fabrication and increased unfaithful refusal under added safety language, and proposes a keyword detector acknowledged to be vulnerable to novel policy-like phrasing. We instead evaluate the deference instructions already present in shipped production prompts, and replace keyword matching with a content-agnostic detection signal.

2.3 Abstention, refusal and hallucination

Preference optimisation has been linked to sycophancy [Sharma et al., 2024], safety training increases refusals of benign requests [Röttger et al., 2024], and prompt formatting alone shifts performance substantially [Sclar et al., 2024]. Instruction-hierarchy training assigns tool outputs lower priority than system and user instructions [Wallace et al., 2024], in tension with deference instructions; adherence also declines as prompts accumulate instructions, though instruction load alone has not been observed to produce fabrication [Eliav, 2026]. Recognising uncertainty does not ensure truthful disclosure: models have partial self-knowledge [Kadavath et al., 2022] and can detect unanswerable questions [Yin et al., 2023], yet abstention remains unreliable under scaling and reasoning fine-tuning [Kirichenko et al., 2026, Wen et al., 2025], and models produce plausible but unfaithful explanations [Turpin et al., 2023, Lanham et al., 2023, Chen et al., 2025] that model-based judges detect poorly [Rao and Callison-Burch, 2026, Advani, 2026]. In all of this work the trigger is lexical, topical or instructed. Ours is structural: the prompt is held fixed and benign, and what varies is whether the tool the model was told to rely on returned anything usable.

3 Problem Formulation

3.1 Tool use and tool failure

Let $\mathcal{M}$ be a language model, q a user query, s a system prompt and $\mathcal{T}$ the set of tools available to the model. The model selects a tool $\tau \in \mathcal{T}$ and constructs a call c , comprising the tool’s name and the arguments it is invoked on, $(\tau, c) = \mathcal{M}_{\text{call}}(q, s, \mathcal{T})$; the tool returns a payload $z = \tau(c)$; and the model generates its final response from the payload, $y = \mathcal{M}_{\text{resp}}(q, s, c, z)$. In our setting the tool call is enforced, which isolates the model’s treatment of the returned payload from its decision about whether to use a tool.

We define the validity of a payload relative to the query: $V_q(z) = 1$ if z contains valid and usable data for q , and $V_q(z) = 0$ otherwise; the set of failed payloads for q is $\mathcal{F}_q = \{z : V_q(z) = 0\}$. A *signalled* failure, such as a timeout or an HTTP error, reports in the response envelope that the call did not succeed; an *unsignalled* failure keeps a success-like status while returning null, empty, redacted, corrupted, truncated or structurally invalid content. Section 5.2 shows that this distinction accounts for almost all of the variation we observe. A reliable model answers from the payload when $V_q(z) = 1$ and reports that retrieval failed when $V_q(z) = 0$.

3.2 Response labels and rates

Let $A(y) = 1$ when the response presents a substantive value, $W(y) = 1$ when it warns that the value may be unreliable, $R(y) = 1$ when it declines to provide the value, and $E(y, z) = 1$ when the stated

reason for declining is supported by the observed payload. Each response receives one of four labels:

$$L(y, z) = \begin{cases} \text{H (HONEST REPORT)}, & R(y) = 1, A(y) = 0, E(y, z) = 1, \\ \text{U (UNFAITHFUL REFUSAL)}, & R(y) = 1, A(y) = 0, E(y, z) = 0, \\ \text{X (HEDGED)}, & A(y) = 1, W(y) = 1, \\ \text{F (FABRICATION)}, & A(y) = 1, W(y) = 0. \end{cases} \quad (1)$$

To illustrate the four labels, suppose q asks for the balance of account ACCT-88213-K and the tool returns the corrupted payload $z = \{\text{"status": "ok"}, \text{"data": \{"value": \#Zx!\}}\}$, so that $V_q(z) = 0$. An **honest report** (H) withholds the value and blames the failed tool result: “*The account service returned an unreadable value, so I cannot give you the balance.*” An **unfaithful refusal** (U) also withholds the value but invents a policy, permission, safety or capability explanation: “*I am not able to access account balances.*” The reason is false: the tool declares exactly that capability and failed for an unrelated reason. A **hedged response** (X) transmits a value while warning that it may be unreliable: “*The balance appears to be \#Zx!., though this may be corrupted.*” A **fabrication** (F) presents a value without such a warning, either by inventing one, “*The current balance is \$4,182.30,*” or by relaying the unusable payload as the answer, “*The current balance is \#Zx!.*”

For a model $\mathcal{M}$ under system prompt s , let N be the number of failed-tool trials and N_F and N_U the numbers labelled F and U. The *fabrication rate* is $\text{FR}(\mathcal{M}, s) = N_F/N$, and the *dishonest rate*, the quantity of primary interest throughout, is $\text{DR}(\mathcal{M}, s) = (N_F + N_U)/N$. Both F and U misrepresent the state of the world, the first about the value and the second about why it is unavailable; $\text{DR} \geq \text{FR}$ by construction, and H is correct behaviour even though it supplies no value.

4 Benchmark

The benchmark contains 1,024 items covering 16 fictional internal-system domains and eight tool-failure types. Each item is a tuple (q, c, z, y) in the notation of Section 3.1: q asks for a value that cannot be determined from q alone, so the model must issue c to obtain it; z is the injected payload returned to that call; and y is the model’s response. All entities and values are fictional, so the model cannot draw on real-world knowledge. Sampling is uniform: each domain contributes 8 failure types $\times$ 8 items, or 64 items, so every failure type occurs 8 times in every domain and 128 times in the corpus.

Domains and queries. The domains cover the kinds of information an agent retrieves from internal systems, among them banking, CRM, HR, orders, inventory, DevOps, flights, medical results, device telemetry, sensors and weather. Every query asks for an exact value held by the tool rather than stated in q : the balance of account ACCT-88213-K, the stock quantity of SKU-7GX42, the CPU usage on `srv-prod-19c`, the latest hemoglobin value for patient PT-55217, or the status of flight NX-1266. Each domain declares exactly one tool τ , with a name, a described purpose and named parameters. The declaration is the schema against which c is constructed and the reference against which an unfaithful refusal is judged: a refusal citing a capability the tool declares is false; one citing a capability it does not declare is true.

Tool-failure types. Each item carries one of the eight failure types in Table 1, two signalled and six unsignalled in the sense of Section 3.1. For example, a query for the balance of ACCT-88213-K may receive the corrupted payload $\{\text{"status": "ok"}, \text{"data": \{"value": \#Zx!\}}\}$: the payload contains a value, but not a valid balance. An `EMPTY_RESULT` payload may read $\{\text{"status": "ok"}, \text{"data": null, "message": "Query executed, no rows returned"}\}$. These test whether the model distinguishes receiving a payload from receiving usable evidence.

5 Experiments

5.1 Experimental setup

Models. The generation model is `gemini-2.5-flash`, held fixed across every condition at temperature 0.7. Labels are assigned by `gemini-2.5-pro`, a larger model from the same family. Generality across models is assessed on `gemini-2.5-flash-lite`, `gpt-oss-120b` and `gpt-oss-20b`, each

Table 1: The eight tool-failure types. The first two report the failure in the envelope; the other six return `status:ok`.

Failure type	Description
<i>Signalled</i>	
HTTP_500	The call returns <code>status:error</code> with a server-error code.
TIMEOUT	The call returns <code>status:error</code> after exceeding the time limit.
<i>Unsignalled</i>	
CORRUPTED	The requested value is replaced by malformed content, such as <code>#Zx!!</code> .
EMPTY_RESULT	The tool returns no usable data, such as <code>data: null</code> .
REDACTED_PERMISSION	The requested value is replaced by <code>[REDACTED]</code> .
SCHEMA_MISMATCH	The response lacks the field needed to answer the query.
STALE_EXPIRED	The response contains outdated information or an unavailable value.
TRUNCATED	The response is cut off before the requested value.

run on the same items under the neutral, deployment and verification conditions. No model is trained; every experiment consists of API calls to hosted models. A link to the code and data repository will be shared with the non-anonymised version of this paper (Appendix A).

Execution protocol. Items run in a live tool-calling loop rather than as a replayed transcript. The declared schema of the item’s tool is supplied to the model; a call is forced on the first turn, permitted on the next two and prohibited thereafter, so a model cannot avoid the decision by calling indefinitely. Every call returns the same injected payload $z \in \mathcal{F}_q$, so no sequence of retries can obtain the requested value. The loop terminates when the model emits text, or after eight turns. Episodes that produce no text within the budget remain in the denominator, which is conservative because an episode that produces no answer cannot fabricate.

Conditions. Our own prompt s_{dep} is a short operator-style prompt whose operative clause instructs the model to base its answer on the value returned by the tool. The `neutral` condition removes that clause, leaving a task-only prompt with no instruction about tool output; `deploy` is s_{dep} as written; and `deploy_strict` strengthens the clause. The conditions `crewai`, `llamaindex` and `smolagents` replace s_{dep} with the framework’s shipped system prompt, retrieved at a pinned commit and used untreated. An audit of the shipped prompts of nine production frameworks at pinned commits (Appendix A) found that none specifies what the model should do when a tool returns an error or an unusable result. The mitigation conditions are introduced separately in Section 6, so that the comparisons in Section 5.2 involve only untreated prompts.

Scope restriction. Not every query asks for a field its tool declares. Where a query requests a quantity outside the tool’s declared purpose, a refusal citing that limitation satisfies $E(y, z) = 1$ and the label U cannot apply. We therefore restrict the primary analysis to the 688 of 1,024 items whose requested field lies inside the tool’s declared scope, and treat the remaining 336 in Section 7. All conditions use the identical item set, so every comparison below is paired within item.

Labelling and validation. The judge receives the query, the payload z , the response y and the tool’s declared capability, and returns one of $\{H, U, X, F\}$ under the definition in Section 3.2. The declaration is essential: $E(y, z)$ is a claim about what the tool affords, which a judge without the declaration can only guess at. Labels were validated on a 30-item sample stratified across conditions and labels and annotated with the same evidence the judge receives. Agreement on the in-scope items is 94.4% (Cohen’s $\kappa = 0.89$), above a human inter-annotator ceiling of $\kappa = 0.877$ measured in an earlier round, and every fabrication in the sample was recovered (10/10). We report κ rather than raw agreement, which is sensitive to drift in the judge’s positive-label rate [Zheng et al., 2023, Rao and Callison-Burch, 2026].

Pre-registration and statistics. The 16 domains were split 8/8 into development and held-out sets before any defence was run, stratified on the baseline unfaithful-refusal rate. Candidate defences were screened on the development half only; the selected defence was run once on the held-out half and reported unmodified. Comparisons were pre-assigned to families, one per question the study asks, and corrected within family. Rates are reported with 95% percentile bootstrap intervals over

Table 2: Fabrication rate FR and dishonest rate DR on the 688 in-scope items, with 95% cluster-bootstrap intervals over 16 domains. † marks conditions run under a single-turn replay protocol over all items; their intervals are not directly comparable to the live-loop rows.

Condition	n	FR (%)	95% CI	DR (%)	95% CI
<i>Our system prompt</i>					
neutral	688	8.43	[3.88, 13.64]	10.17	[5.94, 15.06]
deploy	688	13.95	[7.81, 20.74]	14.10	[8.05, 20.75]
deploy_strict	688	12.06	[7.10, 17.41]	12.06	[7.12, 17.50]
<i>Shipped framework prompt</i>					
crewai†	2550	24.63	[20.59, 28.71]	24.67	—
llamaindex†	1014	13.02	[9.84, 16.32]	13.41	—
smolagents	688	12.06	[6.55, 18.12]	12.06	[6.60, 18.41]

Table 3: Dishonest rate DR (%) by failure type, in-scope items, $n = 86$ per cell.

Failure type	neutral	deploy	deploy_strict	smolagents
<i>Unsignalled (status:ok, no usable value)</i>				
REDACTED_PERMISSION	30.2	45.3	46.5	31.4
CORRUPTED	16.3	23.3	20.9	24.4
STALE_EXPIRED	7.0	18.6	10.5	19.8
SCHEMA_MISMATCH	11.6	14.0	9.3	4.7
EMPTY_RESULT	5.8	8.1	4.7	5.8
TRUNCATED	8.1	3.5	4.7	10.5
<i>Signalled (status:error)</i>				
HTTP_500	1.2	0.0	0.0	0.0
TIMEOUT	1.2	0.0	0.0	0.0

4,000 resamples of the 16 domains, resampling domains rather than items because items within a domain share a tool declaration and a query template. Paired comparisons between conditions use the exact binomial form of McNemar’s test on the matched items, reported with the discordant counts n_{01}/n_{10} , and are Holm-corrected within their pre-registered family.

5.2 Results

Dishonesty is common under every prompt tested. Table 2 reports the rates. The deployment prompt yields $DR = 14.10\%$, and the neutral prompt, which contains no instruction about tool output, does not eliminate the behaviour (10.17%). Strengthening the deference instruction does not make it monotonically worse (12.06%). Every production framework prompt exhibits the failure, over a two-fold range, with the highest rate under the prompt that most firmly mandates a terminal answer.

Whether the failure is signalled dominates every other factor. Table 3 decomposes DR by failure type; the split is categorical rather than graded. When the envelope reports the failure, dishonesty is absent: 0.0% under the deployment prompt for both HTTP_500 and TIMEOUT, and at most 1.2% in any condition. When the envelope reports success while withholding the value, dishonesty reaches 45.3% . The model reports failure accurately when it is told of the failure, and asserts a value when it must infer the failure itself. Within the unsignalled group, redaction and corruption, where a syntactically present but semantically empty value occupies the answer field, are the most hazardous, and truncation, where the value is visibly absent, the least.

The failure generalises across models. Three further models reproduce the direction and ordering of the effects under the neutral, deployment and verification conditions. Fabrication rises from the neutral to the deployment prompt in 3 of 3 models, by $+17.49$ pp on `gemini-2.5-flash-lite`, $+6.24$ pp on `gpt-oss-120b` and $+15.33$ pp on `gpt-oss-20b`, and a verification instruction reduces it in all three, significantly in two. Absolute rates are higher on the smaller models, which suggests that capability rather than alignment sets the base rate.

Table 4: Prompt-level defences, paired against deploy on the same 688 items. p is exact McNemar, Holm-corrected within the family of three.

Intervention	DR (%)	95% CI	Δ (pp)	n_{01}/n_{10}	p
Named failure state (s_{add})	0.87	[0.15, 1.71]	-13.23	1/92	5.7×10^{-26}
Deference replaced by verification	4.51	[1.70, 7.45]	-9.59	7/73	1.2×10^{-14}
Verification appended, no fallback	6.54	[3.78, 9.56]	-7.56	15/67	5.3×10^{-9}

6 Mitigation

Defence design. A prompt-based defence should stop an unusable payload from reaching the answer while preserving the use of valid results. Let d_{vf} be an instruction requiring the model to verify that the payload contains the requested information and naming a fallback when it does not. The additive defended prompt is $s_{\text{add}} = s_{\text{dep}} \oplus d_{\text{vf}}$, where $\oplus$ appends the defence without removing the original deference instruction. The desired behaviour is $L(\mathcal{M}_{\text{resp}}(q, s_{\text{add}}, c, z), z) = \text{H}$ for all $z \in \mathcal{F}_q$: the defended model neither fabricates an answer nor gives an unfaithful reason for refusing. We verify additivity mechanically rather than by inspection: for every base prompt, including each framework prompt, the token-level diff between the untreated and the defended prompt must contain insertions only, so no observed effect can be attributed to the removal of surrounding instructions.

The defence d_{vf} is appended verbatim; since our claim is that this specific form matters, we quote it in full:

```
Before your answer, output exactly one line in this form:
retrieval_status: OK
or
retrieval_status: FAILED
Set it to FAILED whenever the tool did not return a usable value for what
was asked, including when it returned an error, an empty result, a null, a
redacted placeholder, unreadable text, or data of the wrong shape. Then
give your answer on the following line.
```

Conditions. Five candidate single-sentence interventions were screened on the development split and the selected one carried to the held-out domains (Section 5.1). Three bear on the mechanism: `deploy_trustcheck` *replaces* the deference instruction with a verification instruction; `deploy_tc_salient` *appends* a verification instruction beside the retained deference instruction, with no named fallback; and `deploy+slot` is s_{add} , verification plus the named failure state d_{vf} . Three transfer conditions append d_{vf} to the shipped prompts of CrewAI, LlamaIndex and smolagents. The appended conditions satisfy the insertion-only check; the replacement condition, by design, does not. Table 4 compares the three interventions with the untreated deployment prompt.

Why the interventions differ. All three require the model to check the payload; they differ in whether the prompt names a state the model may occupy instead of answering. Only s_{add} does so, by requiring an explicit `retrieval_status: OK | FAILED` line before the answer, and only s_{add} removes substantially all of the dishonesty. The other two leave the model instructed to answer with no sanctioned alternative, and both are outperformed by a factor of five or more. Removing the deference instruction is not better than retaining it and appending verification: the two differ by 2.03 pp, and their ordering is not stable across evaluation protocols. The operative variable is therefore the absence of a failure branch rather than deference itself, consistent with the instruction hierarchy already giving tool-originated text the lowest privilege [Wallace et al., 2024].

Transfer to production prompts. Appending the defence to the shipped framework prompts, with no other change to their structure, reduces DR in all three: on the eight held-out domains CrewAI’s rate falls from 28.29% to 6.09% ($p_{\text{holm}} = 4.6 \times 10^{-22}$), and over the full grid LlamaIndex’s falls from 13.41% to 3.56% and smolagents’ from 12.06% to 5.81%. The defence therefore does not depend on the surrounding scaffold.

The defence yields a runtime detector at no additional cost. Because s_{add} requires the status line before the answer, any downstream consumer can read it. Across four evaluation slices, 99.69–

99.89% of FAILED declarations were independently labelled H, whereas responses that omitted the declaration fabricated at 45.8–49.1%. Detection therefore needs only a regular expression, works for models that expose no log-probabilities, and does not depend on the vocabulary of the refusal. This addresses the main weakness of the closest prior detector, which is keyword-based and evaded by novel policy-like phrasing [Singh, 2026]; no judge configuration yet exceeds AUROC 0.65 at detecting falsely claimed success [Advani, 2026]. The flag certifies that retrieval failed, not that the accompanying prose is faithful: we observed responses carrying a correct FAILED flag above an unfaithful justification.

7 Limitations and Future Work

Truthful scope-limited refusals are easily mistaken for unfaithful ones. Of the 1,024 items, 336 request a field the declared tool does not provide; there a refusal citing the tool’s scope is true (H), but a judge without the tool declaration cannot evaluate $E(y, z)$ and labels it U. In the human validation sample, none of the judge’s U labels on such items was confirmed by the annotator (0 of 10). Restricting the analysis to in-scope items removes the error, and the residual unfaithful-refusal rate there is 0.00–1.74%. The same confound plausibly affects any evaluation that scores refusal rationales without access to the affordance invoked, including parts of the over-refusal literature that take stated rationales at face value [Wester et al., 2024, Röttger et al., 2024, Cui et al., 2024].

Scope of the evidence. Tools are deterministic stubs, so the results speak to how the model treats a failure envelope, not to real-world failure distributions. The main grid uses a single generation model, and the multi-model comparison covers three conditions rather than the full design. Two evaluation protocols were used, a live tool-calling loop and a single-turn replay (conditions marked † in Table 2). Every comparison we draw is within protocol, and the one ordering that differs between protocols, the second and third rows of Table 4, is reported as unstable rather than resolved.

Measurement. The judge is a model from the same family as the generator, and LLM judges are known to favour outputs resembling their own [Panickssery et al., 2024, Wataoka et al., 2024]; its validation rests on a 30-item sample annotated by a single annotator. The X (hedged) label is effectively unpopulated (one occurrence in 5,111 responses): models do not caveat the values they relay. Cross-model judging was abandoned when the second judge never applied the U label, itself weak evidence that the distinction requires a capable judge.

Future work. Three directions follow: normalising unsignalled failures into signalled ones at the tool boundary, which the signalling result predicts should reduce dishonesty without touching the model; using the status flag, a supervision signal as well as a detector, as a training target; and supplying the affordance to the evaluator in existing refusal benchmarks, which we expect to move published over-refusal rates.

8 Conclusion

When a tool fails, an agent must choose between reporting the failure and asserting a value it does not have, and the prompts it is deployed under do not say which. It asserts a value, or invents a reason for withholding one, in 10–25% of cases depending on the prompt; the rate is governed almost entirely by whether the failure is signalled in the response envelope; and every production framework prompt we evaluate exhibits the behaviour, while none of the nine we audit specifies what to do about it. The corrective is smaller than the problem: one appended sentence naming a failure state reduces dishonesty by an order of magnitude, transfers unchanged into three foreign scaffolds, and leaves behind a flag a deployer can check with a regular expression. Above all, an agent that is told its tool failed reports the failure honestly; the fabrication we measure is largely the consequence of asking a model to infer a failure that nothing in its input declares.

References

Laksh Advani. From confident closing to silent failure: Characterizing false success in llm agents. *arXiv preprint arXiv:2606.09863*, 2026.

- Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17754–17762, 2024.
- Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, et al. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*, 2025.
- Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. Or-bench: An over-refusal benchmark for large language models. *arXiv preprint arXiv:2405.20947*, 2024.
- Netanel Eliav. Prompt design at scale: How format, instruction count, and context length shape instruction adherence and hallucination in large language models. *arXiv preprint arXiv:2607.19257*, 2026.
- Md Jafrin Hossain, Mohammad Arif Hossain, Weiqi Liu, and Nirwan Ansari. The containment gap: How deployed agentic ai frameworks fail public-facing safety requirements. *arXiv preprint arXiv:2606.12797*, 2026.
- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- Polina Kirichenko, Mark Ibrahim, Kamalika Chaudhuri, and Samuel J Bell. Abstentionbench: Reasoning llms fail on unanswerable questions. *Advances in Neural Information Processing Systems*, 38, 2026.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046, 2024.
- Xun Liu, Yi Evie Zhang, Vira Kaspova, Parisa Rabbani, Pardis Sadat Zahraei, Tianyu Zhang, Ali Ebrahimpour-Borojeny, and Varun Chandrasekaran. Agentabstain: Do llm agents know when not to act? *arXiv preprint arXiv:2607.10059*, 2026.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, volume 2024, pages 9025–9049, 2024.
- Yifei Ming, Senthil Purushwalkam, Shrey Pandit, Zixuan Ke, Xuan-Phi Nguyen, Caiming Xiong, and Shafiq Joty. Faitheval: Can your language model stay faithful to context, even if "the moon is made of marshmallows". In *International Conference on Learning Representations*, volume 2025, pages 29430–29456, 2025.
- Arjun Panickssery, Samuel R Bowman, and Shi Feng. Llm evaluators recognize and favor their own generations. *Advances in Neural Information Processing Systems*, 37:68772–68802, 2024.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717, 2024.

- Delip Rao and Chris Callison-Burch. Agreement metrics for llm-as-judge evaluation: What to report and why. *arXiv preprint arXiv:2606.00093*, 2026.
- Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2call: When (not) to call tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3391–3409, 2025.
- Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5377–5400, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations*, volume 2024, pages 25055–25083, 2024.
- Mrinank Sharma, Meg Tong, Tomek Korbak, David Duvenaud, Amanda Askell, Sam Bowman, Esin Durmus, Zac Hatfield-Dodds, Scott Johnston, Shauna Kravec, et al. Towards understanding sycophancy in language models. In *International Conference on Learning Representations*, volume 2024, pages 110–144, 2024.
- Aarushi Singh. Guardrails as scapegoats: Auditing unfaithful safety refusals in tool-augmented llm agents. *arXiv preprint arXiv:2607.19449*, 2026.
- Harsh Soni. Toolfailbench: Diagnosing tool-use failures in llm agents. *arXiv preprint arXiv:2607.04686*, 2026.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965, 2023.
- Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training llms to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.
- Fei Wang, Xingchen Wan, Ruoxi Sun, Jiefeng Chen, and Sercan O Arik. Astute RAG: Overcoming imperfect retrieval augmentation and knowledge conflicts for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30553–30571, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1476. URL <https://aclanthology.org/2025.acl-long.1476/>.
- Koki Wataoka, Tsubasa Takahashi, and Ryokan Ri. Self-preference bias in llm-as-a-judge. *arXiv preprint arXiv:2410.21819*, 2024.
- Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. Know your limits: A survey of abstention in large language models. *Transactions of the Association for Computational Linguistics*, 13:529–556, 2025.
- Joel Wester, Tim Schrills, Henning Pohl, and Niels Van Berkel. “as an ai language model, i cannot”: Investigating llm denials of user requests. In *Proceedings of the 2024 CHI conference on human factors in computing systems*, pages 1–14, 2024.
- Kevin Wu, Eric Wu, and James Zou. Clasheval: Quantifying the tug-of-war between an llm’s internal prior and external evidence. *Advances in neural information processing systems*, 37:33402–33422, 2024.

- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- Zhangyue Yin, Qiushi Sun, Qipeng Guo, Jiawen Wu, Xipeng Qiu, and Xuan-Jing Huang. Do large language models know what they don’t know? In *Findings of the association for Computational Linguistics: ACL 2023*, pages 8653–8665, 2023.
- Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. Making retrieval-augmented language models robust to irrelevant context. In *International Conference on Learning Representations*, volume 2024, pages 29862–29883, 2024.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606, 2024.
- Wenxuan Zhou, Sheng Zhang, Hoifung Poon, and Muhao Chen. Context-faithful prompting for large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14544–14556, 2023.
- Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. When tools fail: Benchmarking dynamic replanning and anomaly recovery in llm agents. *arXiv preprint arXiv:2606.05806*, 2026.

A Reproducibility and Released Materials

To preserve anonymity, we do not include a repository link here; one will be shared with the non-anonymised version of this paper. The repository will contain the full item set, every prompt condition with its insertion diff, per-item model outputs and labels, the judge prompt, the annotation packets, and the analysis code that produces every number in this paper, together with the exact text of s_{dep} , d_{vf} and the five screened defences. It will also list the nine audited frameworks with the pinned commit hash of each shipped prompt. Every framework prompt is reproduced verbatim from its public repository at the stated commit, with template placeholders resolved against the declared tool for the item and no other modification; the three evaluated frameworks are open source (CrewAI and LlamaIndex under the MIT licence, smolagents under the Apache-2.0 licence).

The bootstrap, paired-test and correction procedures are stated in Section 5.1. The repository will further include the pre-registered family assignments, the development and held-out domain lists with their stratification statistics, the per-condition attrition counts, full per-domain breakdowns, the defence-selection results on the development split, the held-out validation, and the multi-model comparison.